AI-Based Traffic Monitoring and Anomaly Detection System

Mini Project | Project Phase – 1 | B.Tech Computer Science Engineering

Submitted by

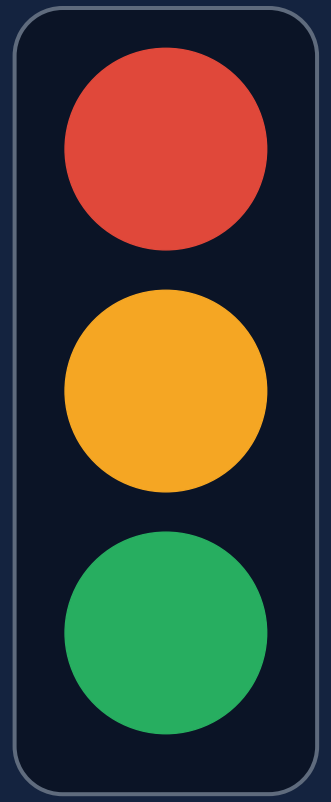
Parth Solanki [202407020494]

Swayam Tanna [202407020495]

Jay Thakkar [202407020496]

Project Mentor: Ms. Pragya Vats

UnitedWorld Institute of Technology, Karnavati University



Presentation Outline

1

Abstract

Summary of the project

2

Introduction

Problem · Objectives · Scope

3

Requirements & Design

Hardware · Software · Architecture · Diagrams

4

Implementation

Technologies · Outputs

5

Conclusion

What this phase achieved

6

Future Scope

Where the project can grow

Abstract



The problem

Watching many CCTV feeds by hand is tiring and slow, so accidents, breakdowns and wrong-way driving are noticed late.



The approach

YOLO vehicle detection, multi-object tracking and a hybrid anomaly detector that mixes rule-based checks with machine learning.



The outcome

A live dashboard with vehicle counts, traffic density and alerts, plus event logs saved with snapshots and timestamps.

Traffic Monitoring | Anomaly Detection | Computer Vision | Deep Learning | YOLO | Object Tracking

Why manual traffic monitoring falls short



Manual monitoring

One operator cannot watch dozens of feeds, so incidents are missed.



Fixed-time signals

Pre-set cycles ignore real queue length and density.



Costly counting

Loops and manual surveys are expensive and give limited data.



Delayed response

Accidents are reported by the public, not detected automatically.



Unused footage

Recordings are watched only after an incident happens.

The need

An automated, camera-based system that monitors traffic continuously and flags abnormal events in real time.

Aim and Objectives

Aim: Design and develop an AI-based system that monitors road traffic from video and automatically detects anomalous events in real time.

1. Study existing traffic monitoring and anomaly detection techniques
2. Detect and classify vehicles with a deep-learning detector
3. Track vehicles and estimate speed and direction
4. Count vehicles; estimate density and congestion
5. Detect stopped, wrong-way, over-speeding and sudden-congestion events
6. Raise real-time alerts; store event logs with snapshots
7. Show everything on an easy operator dashboard

Scope of the Project

In scope

- Live camera streams (RTSP / webcam) and recorded video
- Detection, classification, tracking and counting of vehicles
- Rule-based and ML-based anomaly detection
- Dashboard with live view, statistics, alerts and history
- Evaluation on public datasets and local road videos

Out of scope

- Automatic number-plate recognition and e-challan
- Direct control of traffic signals or control-room hardware
- Large-scale multi-city deployment and edge optimisation

Limitations: accuracy depends on camera angle, lighting, weather and occlusion; speed estimation needs a fixed, calibrated camera.

System Requirements

Hardware

Component	Minimum	Recommended
Processor	Core i5 (8th Gen) / Ryzen 5	Core i7 / Ryzen 7 or higher
RAM	8 GB	16 GB or more
GPU	Optional (slow on CPU)	NVIDIA, 4 GB+ VRAM (CUDA)
Storage	256 GB SSD	512 GB SSD
Camera	720p webcam / IP camera	1080p IP camera, RTSP
Network	Broadband	Wired LAN for IP cameras

Software

Language: Python 3.10+

Deep learning: PyTorch + Ultralytics YOLO

Vision: OpenCV

ML / data: NumPy, Pandas, Scikit-learn

Tracking: ByteTrack / DeepSORT

Dashboard: Streamlit or Flask

Database: SQLite

OS: Windows 10/11 or Ubuntu 22.04

System Architecture: four layers

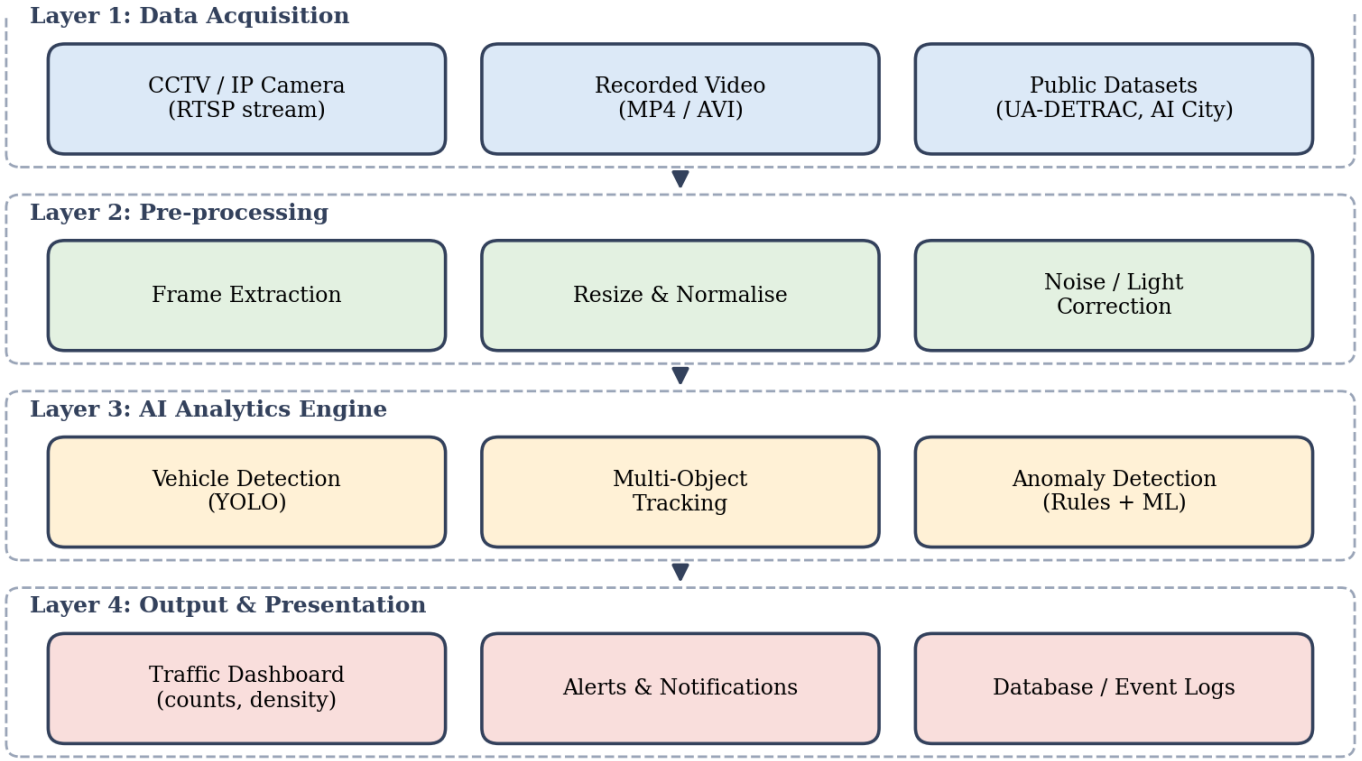


Fig. 1: Layered System Architecture of the AI-Based Traffic Monitoring and Anomaly Detection System

1

Data Acquisition

CCTV / IP cameras, video files and public datasets

2

Pre-processing

Frame extraction, resize, normalise, light correction

3

AI Analytics Engine

YOLO detection, tracking and anomaly detection

4

Output & Presentation

Dashboard, alerts and event database

How each video frame is processed

1

Capture & pre-process

Frames from camera or file, resized and normalised

2

Detect

YOLO gives boxes, vehicle classes and confidence scores

3

Track

ByteTrack / DeepSORT keep one unique ID per vehicle

4

Analyse

Speed, direction, dwell time and lane density

5

Alert

Anomaly found: snapshot, log entry, dashboard update

Real-world speed comes from a perspective (homography) calibration.

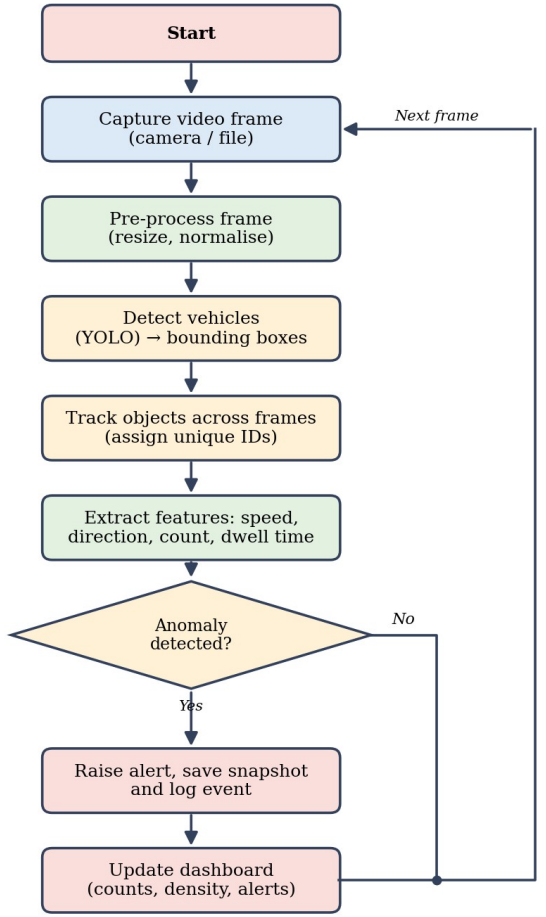


Fig. 2: Flowchart of the Processing Pipeline

Hybrid anomaly detection

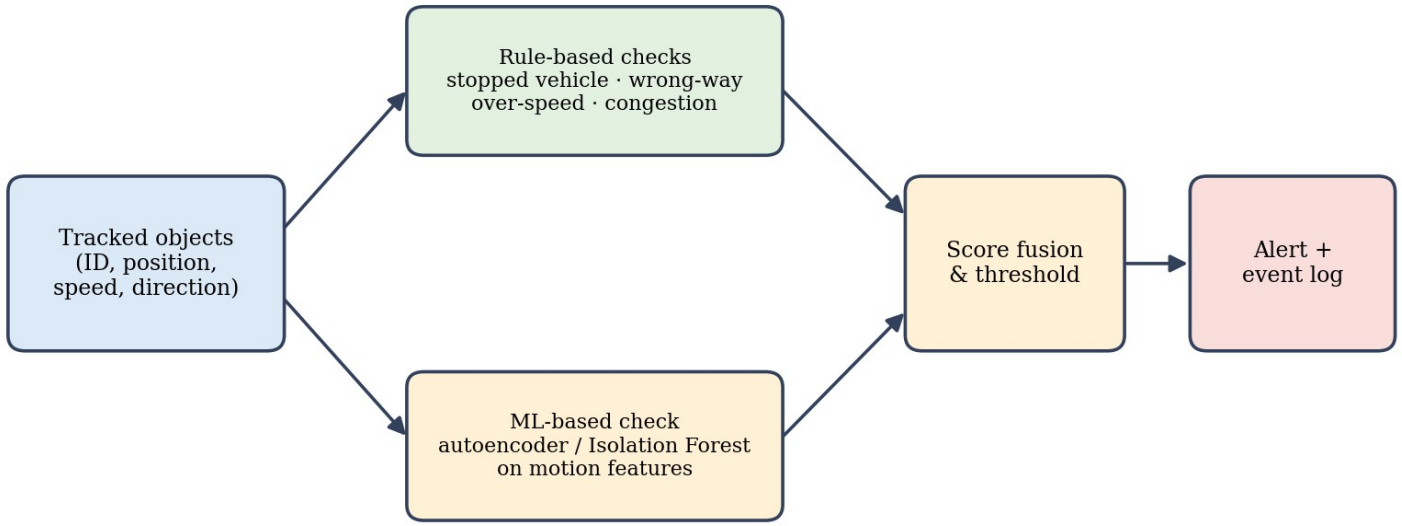


Fig. 4: Hybrid Anomaly Detection Module

Rules

Explainable, no training data needed

Machine learning

Autoencoder / Isolation Forest finds unseen behaviour

Score fusion

Combining both reduces false alarms

Anomalies covered

- Stopped vehicle
- Wrong-way driving
- Over-speeding
- Sudden congestion
- Pedestrian on road

Use case diagram: who uses the system

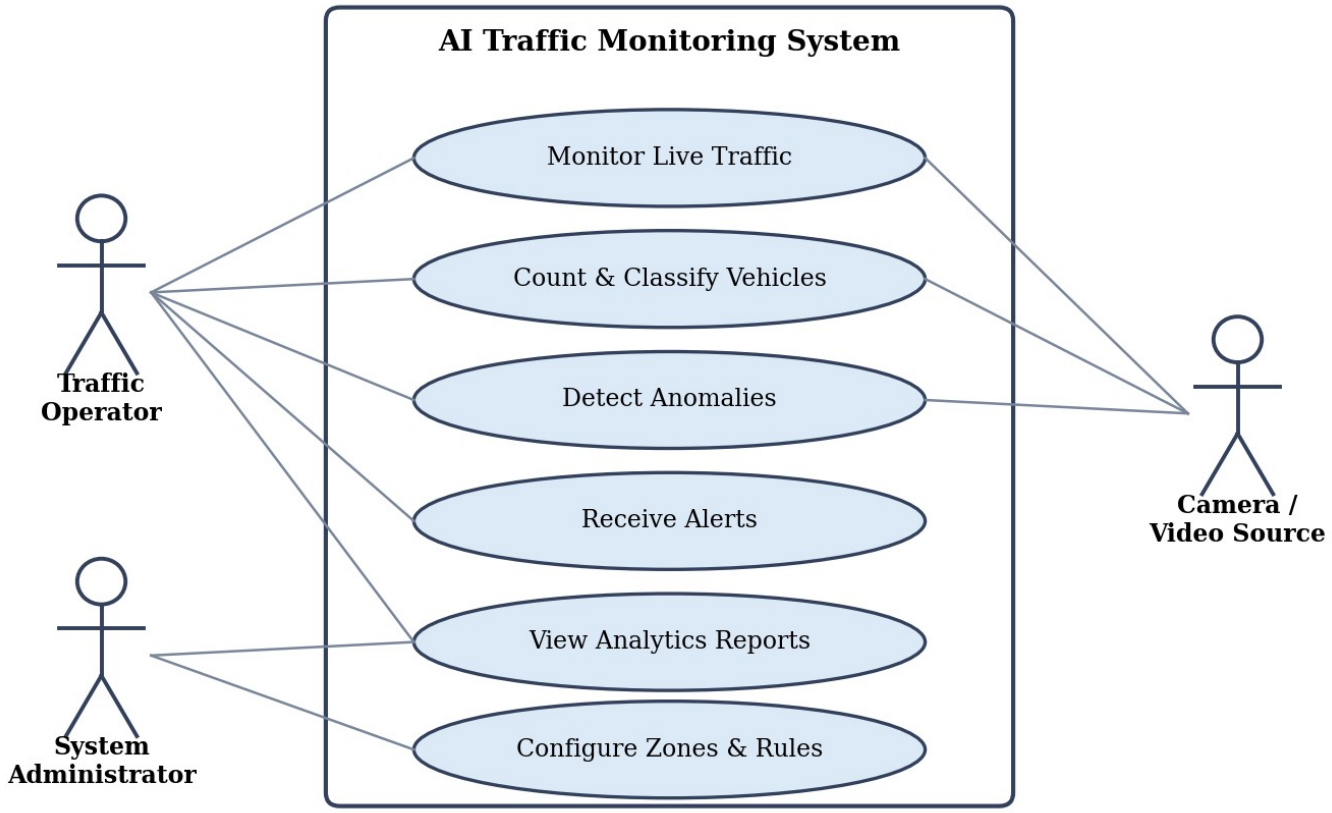


Fig. 3: Use Case Diagram

Traffic operator

Monitors live traffic, views analytics and receives alerts

System administrator

Configures zones, lanes, thresholds and rules

Camera / video source

Supplies live streams or recorded video to the system

Technologies Used

Python
Main language for the full pipeline

OpenCV
Video capture, zones and overlays

PyTorch + YOLO
Vehicle detection and classification

ByteTrack / DeepSORT
Multi-object tracking with IDs

Scikit-learn
Isolation Forest for ML anomaly scores

Streamlit / Flask
Dashboard for live view and alerts

SQLite
Event logs, counts, alert history

Colab + GitHub
GPU training and version control

Datasets: UA-DETRAC (detection and tracking) · COCO (pre-trained weights) · AI City Challenge (anomaly videos) · local road videos

Implementation plan

1

Collect & prepare data

Convert labels to YOLO format

2

Fine-tune YOLO

Train on the required vehicle classes

3

Integrate tracker

Check IDs stay stable under occlusion

4

Define zones & lanes

Counting lines and lane directions in config

5

Build anomaly checks

Rule-based checks plus ML model

6

Build dashboard & DB

Connect alerts and event storage

7

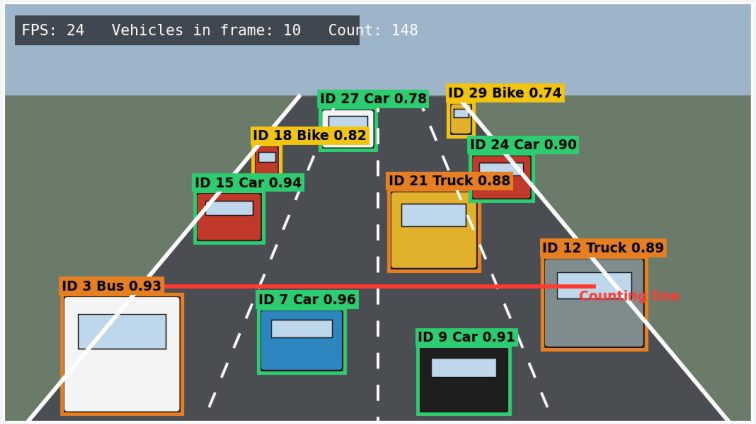
Test and tune

Different light, angles and density

Phase 2

Full implementation, training and testing on real traffic videos

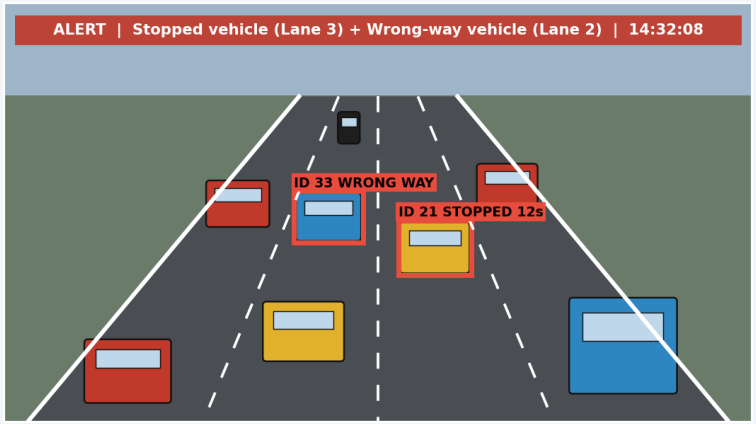
Expected outputs



Detection with boxes, classes and tracking IDs



Dashboard: counts, speed, class-wise count, lane density



Alert: stopped vehicle and wrong-way vehicle

Illustrative output screens of the proposed system.

How it will be evaluated

mAP | MOTA / IDF1 | Precision · Recall · F1 | False-alarm rate | FPS

Conclusion

Phase 1 delivered a complete design for an AI-based traffic monitoring and anomaly detection system: YOLO detection, multi-object tracking and hybrid anomaly detection, supported by a dashboard and an event database.



Modular design

Four clear layers that are easy to build, test and extend.



Affordable

Open-source tools on a normal GPU-enabled computer.



Faster response

Less dependence on operators and earlier incident detection.

Next: full implementation, training, testing and tuning on real traffic videos.

Future Scope



Number-plate recognition

Identify vehicles involved in violations



Violation detection

Helmet, seat-belt, red-light for e-challan



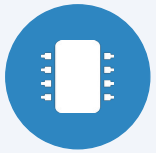
Accident detection

Severity estimation with video transformers



Adaptive signals

Reinforcement learning from live density



Edge deployment

Jetson / Raspberry Pi with TensorRT



Multi-camera tracking

Follow vehicles across junctions



Traffic prediction

LSTM / graph networks forecast congestion



Mobile app and cloud

Alerts to police, ambulance and public



Robustness

Night, fog, rain and heavy occlusion



Thank You

Questions?

Parth Solanki · Swayam Tanna · Jay Thakkar | Mentor: Ms. Pragya Vats